

Lcstrs_of_ocaml *A translator from OCaml to LCSTRS
for termination analysis*

Nikos Liakas StudentNo: 13908640

11 September 2025

School of Computing and
Mathematical Sciences
Birkbeck, University of
London

Supervisor:
Dr. Carsten Fuhs

Academic Declaration:

This report is the result of my own work except where explicitly indicated in the text. I give my permission for it to be submitted to the TURNITIN Plagiarism Detection Service. I have read and understood the sections on plagiarism in the College website and the [Policy and Procedures on academic integrity](#).

The report may be freely copied and distributed provided the source is explicitly acknowledged.

Abstract

Automatic translator from OCaml to Logically Constrained Simply-typed Term Rewrite System expressions. Used for termination analysis with the program analyser **cora**, that parses Term Rewrite System expressions and applies proofs to output of translator. Report contains research on static analysis, term rewrite systems and critical evaluation of the development of a translator for the use of program verification.

Contents

List of Figures

1 Introduction

How does the programmer know that a program satisfies a particular property? By conducting some sort of program analysis. Throughout the history of computer science, many different methods have been used to answer this question.

Like many other sciences, tests have been thought of as an incremental method of stating explicit expectations about a property of a programming. The program would execute and each test would allow the programmer to confirm or reject the expectation formalised in that test.

But as programs became even trivially more complex, the execution paths necessitated orders of magnitude more tests to determine that a particular program is satisfied of a particular property. Luckily, parallel to the development of programming languages, computer science was developing methods to analyse programs without running them. These methods are grouped as *static analysis* methods. Static analysis is a form of program analysis using only the syntactic expression of the program, or a compiled version of it to determine if a program possesses a property. This is defined in contrast to *dynamic analysis* where a program needs to execute to determine the presence or absence of the property in the program.

This report focuses on automating a form of static analysis called *termination analysis*. As will be shown, answering the question of will a program terminate on a set of inputs, is not a trivial problem. The static analysis method used for this is called a Term Rewrite System (TRS). This formalism is used as an input to a number of provers and satisfiability tools to determine particular properties of programs. The shortcoming with this approach is that the source-code needs to be translated into the TRS expression, before a tool can accept it. The tool that will consume the target of the project's translator is `cora` [?].

The project focuses on the development of TRS from its early development as an expression of equational logic, through to its development to a syntactic expression that is more proximate to programming languages by expanding its syntactic capability to more clearly mirror those languages. Many modern programming languages are *Turing complete*, which means, roughly, any system that can perform any computation that can be executed by an algorithm. They are deemed so with the added enablement that they have infinite time and memory. The TRS formalism is Turing complete. So development of the formalism, as will be shown below, does not imply a semantic change in the system's ability to compute any defined computable algorithm.

`Lcstrs_of_ocaml` automates this translation from OCaml source-code to *Logically Constrained Term Rewrite System* (LCSTRS) expressions. `cora` can parse and analyse different TRS formalisms, denoted by the file extension. This project is limited to LCSTRS expressions.

Summary. The report begins with a first section on the research of the LCSTRS formalism. It situates the LCSTRS within the static analysis context, the use of automation and approximation within this process and how this is used to verify termination by `cora`. The next section covers the design of `Lcstrs_of_ocaml`. In

this section the architecture and design of each program module are examined, followed by the orchestrator and program flow. The next section focuses on implementation of the program. The command line interface, type table, rules and signatures as well as the development methodology are covered in this section. Testing and evaluation are presented in the subsequent section covering the unit tests, validation tests for `cora` input and an evaluation of the different stated aims of the program from the proposal. Finally, a critical review is the final section of the report. In it, the success or failure is assessed of the proposal's stated aims, and a discussion of the type-system is conducted with reference to an alternative path for a component's development, among other topics.

2 Research: LCSTRS formalism

This section focuses on the conceptual landscape of LCSTRSs by first introducing the reader to static analysis, the LCSTRS formalism and finally, the formalism's concrete expression and syntax in `cora`.

2.1 Static Analysis

Static Analysis is the "...technique for program-level analysis that approximates ...semantic properties of programs before their execution" [?, 17]. The opposite of static analysis is *dynamic analysis*, which determine properties of programs by executing the code under analysis.

Let's look at the process steps of a well known form of dynamic analysis, M Fowler's *Refactoring* [?]. The goal of Refactoring is to change a program to a *roughly* semantically equivalent program, with particular design and style principles as goals. One step in this process is the definition of a 'test-harness'. This is a set of tests that represent the programmer's understanding of the semantic state of the program, how it is expected to behave, according to the programmer. When each refactor occurs, these tests are run to ensure that the tested semantic local properties are still valid, in relation to the source-code. The programmer needs to define these tests, and this process is a necessary part of the refactoring process.

`Lcstrs_of_ocaml`'s place within the static analysis chain is that of automation. The program *translates* source-code into an input that `cora` can analyse for termination. `Lcstrs_of_ocaml` and `cora` represent the enabling characteristics of static analysis as a viable analysis path for the programmer.

2.1.1 Automation and Approximation

Representing a program in mathematical formalisms whose properties can be reasoned about using proofs is at least as old as Turing and Church's proofs for the undecidability of the universal *halting* problem [?] [?]. Is there a way to determine in a limited set of programs and inputs that a given program halts? This way is provided by *automated* termination analysis, through **Automation** and **Approximation**.

1. **Automation:** analysis that possesses this characteristic does not require user input. `Lcstrs_of_ocaml` automates OCaml’s input into `cora` without intervention from the programmer to define the terms, rules and signatures for the term rewrite system expression used as an input.
2. **Approximation:** approximations can be declared for programs to enable decidability of termination, as the universal halting problem is proven undecidable. Analysers are categorised in their approximation in relation to *soundness* and *completeness*, see ??.

Example 2.1. Soundness and Completeness The following example is that of a theoretical program analyser called `analysis` from Rival and Yi’s *Introduction to Static Analysis* (2020).

If `analysis` were perfectly accurate universally it would satisfy the equivalence property:

$$\text{for every program } p \in \mathbb{L}, \text{analysis}(p) = \text{true} \iff p \text{ satisfies } \mathcal{P}$$

This states that if any program p , expressed as a subset of a language $\mathbb{L}$, is input into the `analysis` tool and that program evaluates to `true` then this is equivalent to p satisfying this property. This is a subtle difference that guarantees two characteristics of the `analysis` tool. First, that it will include all programs that satisfy a given property **and** second, that it exclude those that do not. When process of determining a property universally is proved to be undecidable one needs to choose one of these two characteristics:

Soundness a sound analyser will guarantee that an input program satisfies a given property.

Completeness a complete analyser will never miss a program that satisfies a given property.

Expressed formally, the equivalence relation above is decomposed into a pair of implications:

$$\begin{cases} \text{for every program } p \in \mathbb{L}, \text{analysis}(p) = \text{true} \Rightarrow p \text{ satisfies } \mathcal{P} \\ \text{for every program } p \in \mathbb{L}, \text{analysis}(p) = \text{true} \Leftarrow p \text{ satisfies } \mathcal{P} \end{cases} \quad (1)$$

[?, 9]

An analyser is *sound* if it satisfies the first implication and *complete* if it satisfies the second. The difference between them is subtle. The approximation provided by a sound analyser will miss valid programs satisfied of property $\mathcal{P}$ by erring on the side of caution and reducing its verification space, even within the possible valid programs in relation to $\mathcal{P}$ space. The goal is to ensure the property $\mathcal{P}$ is satisfied.

The approximation provided by a complete analyser will falsely output that a given program satisfies $\mathcal{P}$ by expanding its verification space beyond the possible ‘valid program in relation to $\mathcal{P}$ ’ space. The goal is to capture all programs that satisfy $\mathcal{P}$.

2.1.2 Static Analysis and LCSTRS

LCSTRS expressions can be *symbolically* executed. `cora` takes TRS expressions as inputs and is able to apply proofs to the expressions, rather than execute them as machine instructions.

The analyser will not guarantee a property that is not present in the program, but it may reject programs that are satisfied of that property. The analyser used for this project (`cora`) responds with ‘YES’, ‘NO’ or ‘MAYBE’, after applying several techniques to determine termination. The analyser `cora` is a **sound** analyser.

Thus, this project aims to ensure the presence of the property of termination if the analyser determines this property to be present. It does not aim to capture all OCaml programs that are possessed of the property of termination.

2.1.3 Automation of the analyser `cora`

`Lcstrs_of_ocaml` extends the capabilities of `cora` by providing an *automated* translation mechanism for termination verification inputs of OCaml. Different translators can exist for different languages. OCaml is the focus of this project.

2.2 LCSTRS

This section describes Term Rewrite System (TRS) expressions and their particular development to the formalism used as a target for `Lcstrs_of_ocaml`, Logically Constrained Simply-typed TRS (LCSTRS) expressions.

2.2.1 The Term Rewrite System

Mathematical formalisms that express the dynamic nature of a programming language have existed since lambda calculus and equational logic notation. Though many precedents could be placed for the origins of TRS, this report will choose Baader and Nipkow’s (1998) monograph [?] as the substantive point at which Term Rewrite formalisms were consolidated into one work, in English. The authors describe the use of TRS expressions as an *Intermediary Representation* (IR) between the source-code of a programming language and its analyser. This syntactic distance allows for proofs developed for TRS expressions to be applied to decide the presence or absence of a given property.

Definition 1. Term Rewrite System (TRS): a set of *directed* replacement rules to a subterm to translate it into another term [?, 1]. This is expressed using a finite formalism of an *abstract reduction system*, which can be infinite. The replacement rules are applied to terms to *reduce* them to an irreducible *normal* form. The nature of these reductions, their applications and evaluations are inputs into proofs that determine properties of programs [?, 8-9].

In ordinary TRSs cited above, $\mathcal{R}$ is a set of rules $\ell \rightarrow r$, and ℓ and r are terms. These substitutions, from one term to another, are called *rewrite* steps.

A sequence of rewrite steps is called a *reduction*. Below we will show an example of an expression in OCaml and its translation into a TRS expression.

```
# 1 + 4
- : int = 5
```

TRS-translation. Natural numbers are represented using the constant 0 and the application of successor functions s . $\mathcal{R}$ contains the following rules:

$$\begin{aligned} x + 0 &\rightarrow x \\ x + s(y) &\rightarrow s(x + y) \end{aligned}$$

One translates 1 and 4 initially with the constant and successor functions. Then we apply the substitution rules to evaluate until we reach a state where no more substitutions are possible. *Terms* in ordinary TRS are built from variables, constant symbols and function symbols.

$$\begin{aligned} s(0) + s(s(s(0))) &\rightarrow \\ s(s(0) + s(s(0))) &\rightarrow \\ s(s(s(0)) + s(s(0))) &\rightarrow \\ s(s(s(s(0))) + s(0)) &\rightarrow \\ s(s(s(s(s(0)))) + 0) &\rightarrow \\ s(s(s(s(s(0))))) & \end{aligned}$$

In the above translation, $+$ is a binary function symbol, s is a unary function symbol, 0 is a constant and x and y are variables.

There are practical limitations with TRS expression analysers. It is possible, though difficult to express non-natural numbers, but syntactically cumbersome. It would be difficult to use ordinary TRS to express integers or floating point numbers [?, 1].

2.2.2 Many-Sorted TRS

The development of the Many-Sorted TRS allowed for more syntactically powerful expressions by introducing many sorts. These restrict terms. The following is the formalism for signature, sorts, variables, terms and typing for the many-sorted TRS.

Definition 2. Many-Sorted TRS

A *signature*, denoted by Σ , is a set of function symbols f , each of which is accompanied by a *sort declaration* in the form $[\iota_1 \times \dots \times \iota_n] \Rightarrow \kappa^1$, elements of the set of all sorts $\mathcal{S}$. An infinite set of variables $\mathcal{V}$ is disjoint from $\mathcal{S}$.

¹ κ is used instead of $\square \Rightarrow \kappa$ for readability.

$Terms(\Sigma, \mathcal{V})$ is a set of *terms* over Σ and $\mathcal{V}$. $\mathcal{V}$ contains any expression s such that some sort ι can be derived through inference.

$$\frac{}{\vdash x : \iota} (x : \iota \in \mathcal{V}) \quad \frac{\vdash s_1 : \iota_1 \cdots \vdash s_n : \iota_n}{\vdash f(s_1, \dots, s_n) : \kappa} (f : [\iota_1 \times \cdots \times \iota_n] \Rightarrow \kappa \in \Sigma)$$

Σ and $\mathcal{V}$ are fixed. For every term s there is a distinct sort ι that can be derived $\vdash s : \iota$ [?, 5].

2.2.3 LCTRS and LCSTRS

LCTRSs. LCTRSs are expanded with the below definition

Definition 3. Signatures for LCTRS

Signature types are introduced that divide the term types that can be expressed.

- Σ is now formed by the union of Σ_{terms} and Σ_{theory} .
- a map $\mathcal{I}$ for each sort ι in Σ_{theory} a mapping $\mathcal{I}_\iota$ is associated with each function symbol.
- a map $\mathcal{J}$ which assigns for each function and sort declaration in Σ_{theory} a function in $\mathcal{I}_{\iota_1} \times \cdots \times \mathcal{I}_{\iota_n} \Rightarrow \mathcal{I}_\kappa$.

Logical terms are members of $Terms(\Sigma_{theory}, \mathcal{V})$ [?, 6].

Example 2.2. This example with $\mathcal{I}_{int} = \mathbb{Z}$ and $\mathcal{I}_{bool} = \mathbb{B}$ could have a Σ_{theory} :

$$\begin{array}{lll} \text{true} : \text{bool} & + : [\text{int} \times \text{int}] \Rightarrow \text{int} & \wedge : [\text{bool} \times \text{bool}] \Rightarrow \text{bool} \\ \text{false} : \text{bool} & \geq : [\text{int} \times \text{int}] \Rightarrow \text{bool} & \neg : [\text{bool}] \Rightarrow \text{bool} \\ n : \text{int}(n \in \mathbb{Z}) & = : [\text{int} \times \text{int}] \Rightarrow \text{bool} & \end{array}$$

Our Σ_{terms} could be:

$$\text{sum} : [\text{int}] \Rightarrow \text{int} \quad \quad \text{n} : \text{int}(\forall n \in \mathbb{Z})$$

[?, 6]

Definition 4. LCTRS

A *logically constrained term rewriting system* (LCTRS) is defined as the pair $(Terms(\Sigma, \mathcal{V}), \rightarrow_{\mathcal{R}})$. $\rightarrow_{\mathcal{R}}$ is a reduction. A *constraint* is a logical term φ with sort **bool**. Its mapping is $\mathcal{I}_{\text{bool}} = \mathbb{B} = \{\top, \perp\}$

The above three definitions give us the minimum required for an LCTRS expression:

- $\mathcal{I}$
- $\mathcal{J}$

- Σ_{theory}
- Σ_{term}
- $\mathcal{R}$

This is a fair amount of notation, so it is best to review a concrete example:

Example 2.3. LCTRS of OCaml

This is a function that calculates the factorial of a natural number:

```
let rec fact n =
  if n > 0 then
    n * fact (n-1)
  else
    1
```

The output from `Lcstrs_of_ocaml` is:

```
fact :: Int -> Int

fact(n) -> n * fact(n - 1) | n > 0
fact(n) -> 1 | n <= 0
```

This is the breakdown of the `fact` function using the formalism defined above:

- **Signature (Σ):** The signature is the union of the theory signature (Σ_{theory}) and the term signature (Σ_{theory}), as stated above.
 - Σ_{theory} includes types and operators on terms like integers and booleans.
 - * Sorts: `Int` and `Bool`
 - * Constants: Integers $n : \text{int}(\forall n \in \mathbb{Z})$ examples 0 and 1
 - * Functions: these functions are used internally by the terms and are not the signature of the terms themselves. `cora` has many of the common theory terms pre-defined like common binary functions (`+`, `-`, `*` etc.); these do not need to be provided by the translator.:
 - Binary functions: `- : [Int → Int] → Int`, `* :: [Int → Int] → Int`
 - Comparison functions: `> :: [Int → Int] → Bool`, `<= :: [Int → Int] → Bool`
 - Σ_{terms} function symbols related to the program itself and to terms whose definition is not already defined in `cora`. These need to be made explicit by the translator.
 - * : Sorts: `Int`
 - * : Functions: `fact :: Int → Int`
 - Interpretation $\mathcal{I}$: this maps Σ_{theory} sort ι to its mathematical domain. This is handled internally by `cora`.

- * $\mathcal{I}_{\text{Int}} = \mathbb{Z}$ the set of all integers.
- * $\mathcal{I}_{\text{Bool}} = \mathbb{B}$ true or false.
- Rule Set $\mathcal{R}$: the directed replacement rules of the finite abstract reduction system. In the example, both rules contain a logical constraint (φ) after the delimiter $|$.

$$\begin{aligned} \text{fact}(n) &\rightarrow n * \text{fact}(n - 1) | n > 0 \\ \text{fact}(n) &\rightarrow 1 | n \leq 0 \end{aligned}$$

- Variables $\mathcal{V}$: these are the variables used in the rules. `cora` attempts to infer these from the term signatures much in the same way that OCaml does. If the term signature declaration is $\text{Int} \rightarrow \text{Int}$ the variable input into the term function should be an integer as bounded by the interpretation.
- * `n` this variable is of the sort `Int`

LCSTRS. *Logically Constrained Simply-typed TRS* (LCSTRS) extend LCTRS to include *higher-order* functions, as well as applicative type declarations. A defining feature of OCaml and other programming languages that implement the functional programming paradigm is the use of higher-order functions, that is functions that can take other functions as input and output another function.

Definition 5. Simply Typed TRS (STRS) Formalism

The LCSTRS formalism is a combination of the STRS formalism with logical constraints. A non-empty set $\mathcal{S}$ of *base types* called *sorts* forms the simple type case. A set $\mathcal{T}$ of simple types over $\mathcal{S}$, produced by the grammar $\mathcal{T} ::= \mathcal{S} \mid (\mathcal{T} \rightarrow \mathcal{T})$. There are two disjoint sets: the set of function symbols $\mathcal{F}$ and variables $\mathcal{V}$. A set of *pre-terms* $\mathfrak{T}$ is generated by the grammar $\mathfrak{T} ::= \mathcal{F} \mid \mathcal{V} \mid (\mathfrak{T}, \mathfrak{T})$.

Example 2.4. Typing for *pre-terms* Application is left-associative for pre-terms: $t_0 t_1 t_2$ can be literally expressed $((t_0 t_1) t_2)$. The typing works as expected for pre-terms.

$$\begin{aligned} t_0 &\text{ has type } A \longrightarrow B \\ t_1 &\text{ has type } A \\ t_0 t_1 &\text{ has type } B \end{aligned}$$

Terms over $\mathcal{F}$ and $\mathcal{V}$ are denoted by $T(\mathcal{F}, \mathcal{V})$. A set of rewrite rules is denoted as $\mathcal{R} \subseteq T(\mathcal{F}, \mathcal{V}) \times T(\mathcal{F}, \mathcal{V})$.

Limitations. Product types, polymorphism and λ abstractions are not offered by this formalism. Guo and Kop expect to adopt other higher-order formalisms in time. [?, 3]

With the above there is enough to define the LCSTRS formalism

Definition 6. A Logically Constrained STRS (LCSTRS) is composed of $\mathcal{S}, \mathcal{S}_\vartheta, \mathcal{F}, \mathcal{S}_\vartheta, \mathcal{V}, (\mathfrak{X}_A), \llbracket \cdot \rrbracket$ and $\mathcal{R}$ where:

1. $\mathcal{S}$ is a set of sorts,
2. $\mathcal{S}_\vartheta \subseteq \mathcal{S}$ is a set of theory sorts, containing `bool`,²
3. $\mathcal{F}$ is a set of function symbols, with each function symbol assigned a simple type over $\mathcal{S}$,
4. $\mathcal{F}_\vartheta \subseteq \mathcal{F}$ is a set of theory symbols each with a theory type over $\mathcal{S}_\vartheta$, $f : \text{bool}$ and $t : \text{bool}$ are elements of $\mathcal{F}_\vartheta$,
5. $\mathcal{V}$ is a set of variables, they are disjoint from $\mathcal{F}$, are assigned a simple type over $\mathcal{S}$ and there are infinitely many variables to which every type is assigned,
6. $(\mathfrak{X}_A)$ is an $\mathcal{S}_\vartheta$ -indexed family of sets such that $\mathfrak{X}_{\text{bool}} = \{0, 1\}$,
7. $\llbracket \cdot \rrbracket$ ³ is an interpretation of theory symbols such that $\llbracket f \rrbracket = 0$ and $\llbracket t \rrbracket = 1$, and
8. $\mathcal{R} \subseteq T(\mathcal{F}, \mathcal{V}) \times T(\mathcal{F}, \mathcal{V}) \times T(\mathcal{F}_\vartheta, \mathcal{V})$ is a set of rewrite rules.

$\mathcal{R}$ alone often denotes the system. [?, 7-8]

Below is an example of OCaml to LCSTRS using the formalism described:

Example 2.5. OCaml to LCSTRS

```

let rec take n l =
  match (n, l) with
  | (0, _) | (_, []) -> []
  | (n, h :: t) -> h :: take (n - 1) t

```

The simplified translation of the function above: [?, 8]

$$\begin{aligned}
& \text{take } n \text{ nil} \rightarrow \text{nil} \\
& \text{take } n \text{ l} \rightarrow \text{nil} & [n \leq 0] \\
& \text{take } n (\text{cons } h \text{ l}) \rightarrow \text{cons } h (\text{take } (n - 1) \text{ l}) & [n > 0]
\end{aligned}$$

where

$$\begin{aligned}
\mathcal{S} &= \mathcal{S}_\vartheta \cup \{\text{intlist}\}, \mathcal{S}_\vartheta = \{\text{bool}, \text{int}\}, \\
\mathcal{F} &= \mathcal{F}_\vartheta \cup \{\text{nil}, \text{cons}, \text{take}\}, \mathcal{F}_\vartheta = \{\leq, >, -, f, t\} \cup \mathbb{Z}, \\
\mathcal{V} &\supseteq \{l, n, h\},
\end{aligned}$$

²The composition of the theory sort `bool` is what allows *constrained* STRS. This is explicit in [?, 7].

³6 and 7 are $\mathcal{S}_\vartheta$ -indexed and $\mathcal{T}_\vartheta$ -indexed respectively. $\llbracket \cdot \rrbracket_A$ assigns to each theory symbol of type A an element of $\mathfrak{X}_A$.

$$\begin{array}{ll}
\leq : \text{int} \rightarrow \text{int} \rightarrow \text{bool}, & > : \text{int} \rightarrow \text{int} \rightarrow \text{bool}, \\
- : \text{int} \rightarrow \text{int} \rightarrow \text{int}, & \text{f} = \text{bool}, \quad \text{t} = \text{bool}, \\
n : \text{int} \text{ for all } n \in \mathbb{Z}, & \text{nil} : \text{intlist}, \\
\text{cons} : \text{int} \rightarrow \text{intlist} \rightarrow \text{intlist}, & \text{take} : \text{int} \rightarrow \text{intlist} \rightarrow \text{intlist}, \\
l : \text{intlist}, & n : \text{int} \text{ and } h : \text{int}
\end{array}$$

For completeness a higher-order function is presented without the theory symbols for `intlist` and `int`.

Example 2.6. `map` in OCaml to LCSTRS

```

let rec map f l =
  match l with
  [] -> []
  | h::t -> f h :: map f t

```

A simplified translation below:

$$\begin{array}{l}
\text{map } f \text{ nil} \rightarrow \text{nil} \\
\text{map } f (h : t) \rightarrow (f h) : \text{map } f : t
\end{array}$$

Note that `map` as declared applies a polymorphic function to a polymorphic list. In `??`, the type limitations are critically evaluated.

2.3 Termination and Verification

Termination analysis is the study of the undecidable *halting* problem mentioned in `??`. It analyzes tools and techniques to work on non-universal decidable determinations. Verificaiton is the activity of determining that a program satisfies a property.

This project's scope is limited to translating OCaml to the LCSTRS expressions that can verify termination programs using `cora`.

2.4 Cora: Rules, Signatures and Term-Rewriting

Term-Rewriting can be expressed in a few ways. The formalism that is the target of our translation is that used as input for the `cora` [?] program analyser. As such, the input is formatted with two components:

Cora: Signature

Signatures are maps with keys being the theory term symbols and their theory sort assignments. The two are delimited by `::`. The theory sort assignments are expressed as left-associative declarations, with the `->` directed operator. A binary operator like `+` in OCaml would have this signature in `cora`:

$$+ :: \text{Int} \rightarrow \text{Int} \rightarrow \text{Int}.$$

Cora: Rules

Rules are composed of a *left-hand term* and its reduced form as a *right-hand term*. The left-hand term needs to declare its signature in `cora` for the analyser to accept the input. Undeclared signatures for each term function symbol will not be accepted.

Example 2.7. Cora Example

This example illustrates the input to `cora` and its output.

- OCaml code:

```
let my_func x =  
  let y = ref 0 in  
  while (x > y) do  
    y := !y + 1  
  done
```

- LCSTRS expression from `Lcstrs_of_ocaml`:

```
my_func :: Int -> A  
l1 :: Int -> Int -> A  
l0 :: Int -> Int -> A  
  
my_func(x) -> l0(x, 0)  
l0(x, y) -> l1(x, y) | x > y  
l1(x, y) -> l0(x, y + 1)
```

- Cora StdOut output:

MAYBE

We consider termination of the LCSTRS with only rule scheme Calc:

```
Signature: l0      :: Int → Int → A  
          l1      :: Int → Int → A  
          my_func :: Int → A
```

```
Rules: my_func(x) → l0(x, 0)  
       l0(x, y) → l1(x, y) | x > y  
       l1(x, y) → l0(x, y + 1)
```

The system is accessible function passing by a sort ordering that equates all sorts.

We start by computing the initial DP problem $D1 = (P1, R, f, c)$, where:

- P1. (1) $my_func\#(x) \rightarrow l0\#(x, 0)$
 (2) $l0\#(x, y) \rightarrow l1\#(x, y) \mid x > y$
 (3) $l1\#(x, y) \rightarrow l0\#(x, y + 1)$

***** We apply the Graph Processor on $D1 = (P1, R, f, c)$.

We compute a graph approximation with the following edges:

1: 2
2: 3
3: 2

There is only one SCC, so all DPs not inside the SCC can be removed.

Processor output: { $D2 = (P2, R, f, c)$ }, where:

P2. (1) $l0\#(x, y) \rightarrow l1\#(x, y) \mid x > y$
(2) $l1\#(x, y) \rightarrow l0\#(x, y + 1)$

***** We apply the Theory Arguments Processor on $D2 = (P2, R, f, c)$.

We use the following theory arguments function:

$l0\# : [1, 2]$
 $l1\# : [1, 2]$

Processor output: { $D3 = (P3, R, f, c)$; $D4 = (P4, R, f, c)$ }, where:

P3. (1) $l0\#(x, y) \rightarrow l1\#(x, y) \mid x > y$
(2) $l1\#(x, y) \rightarrow l0\#(x, y + 1) \{ x, y \}$

P4. (1) $l1\#(x, y) \rightarrow l0\#(x, y + 1)$

***** No progress could be made on DP problem $D3 = (P3, R, f, c)$.

The example above illustrates *cora*'s execution and some provers and solvers applied to an LCSTRS expression.

2.4.1 Cora: Output

cora outputs its analysis as shown above with a determination of YES, NO, MAYBE for the satisfaction of the property of termination on the program. For more on *cora*'s [formalism](#), follow the link [?].

3 Design

This section focuses on the design of *Lcstrs_of_ocaml*. It begins with a high-level description of the program's architecture, continues with an extended examination of the 'translator' through its two evaluation paths. Each path is determined by the type of *cora* input that is needed namely, *signatures* and *rules* [?].

3.1 Architecture

LCSTRS of OCaml consists of three components. The first is the Orchestrator. A user interfaces via the command line and runs the program's executable with flags to process a string of OCaml or an `.ml` file of OCaml. This component executes the second component, the Translator, with a lexer buffer as input. The Orchestrator either provides a LCSTRS artefact as output from the Translator or outputs errors to the user. Finally, the `cora` compatible output is provided to the user via the StdOut channel along with the LCSTRS artefact.

3.2 Translator

The translator is a rudimentary compiler from OCaml source-code to LCSTRS code. This can be represented as a directed acyclic graph. The following sections describe the design of each process step in `??`. The green represents the path for OCaml source-code to rule translations. The LCSTRS signatures are constructed via the orange path with a symbol table. Finally, these are pretty printed into `.lcstrs` artefacts and returned to the Orchestrator.

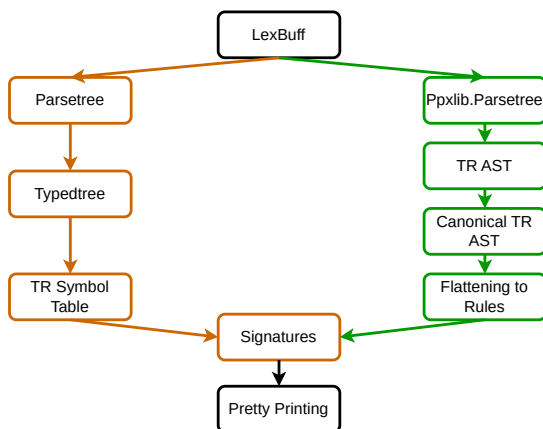


Figure 1: Translator process graph

3.2.1 Lexbuf

Lexbuf is the data structure used as input for the translator. It is an OCaml record with fields that are used by the `Parsetree` module to parse the lexer output into a `Parsetree` type value.

3.2.2 Parsetree and Typedtree

The OCaml compiler has a frontend and backend as shown in `??`.

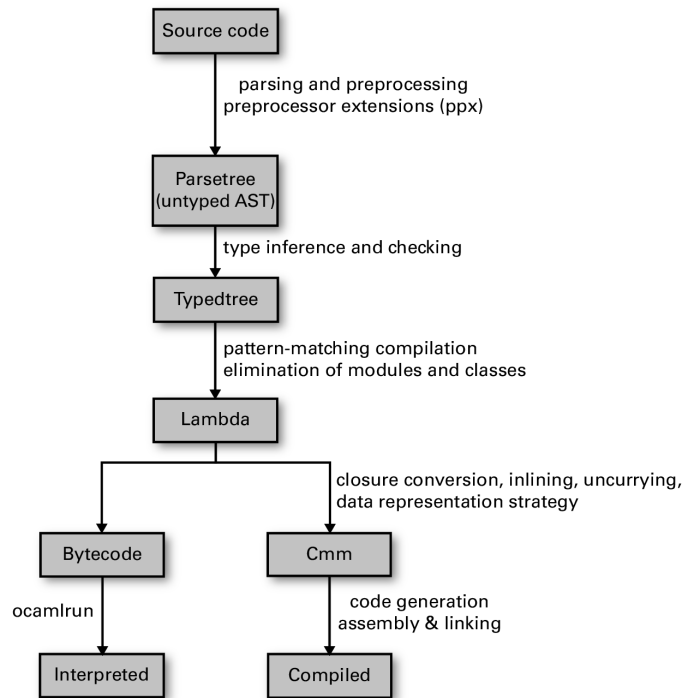


Figure 2: OCaml compiler toolchain [?]

1. Frontend:

- 1.1. **Parsetree**: module used as an OCaml type used for later compilation, before typing
- 1.2. **Typedtree**: module used as an OCaml type that performs OCaml's type inference

2. Backend:

- 2.1. **Lambda**: Creation of optimised pattern-matching automata [?]
- 2.2. **Bytecode path**: bytecode interpreted by the the ocaml virtual machine
- 2.3. **Native code path**: architecture dependent assembly code

Parsetree. This module provides an interface with the parsed AST of the OCaml source-code. It is the foundational structure for pre-processing in OCaml and the first stage after lexing. An example is below of a portion of the representation of an if-then-else expression.

Example 3.1. ParsetreeIllustration

This function takes an integer `n` and returns a string indicating direction.

```

let choice n =
  if n > 0 then
    "up"
  else
    "down"

```

The `Parsetreetype` is a large complex type. Here is a portion of the abstract syntax tree produced:

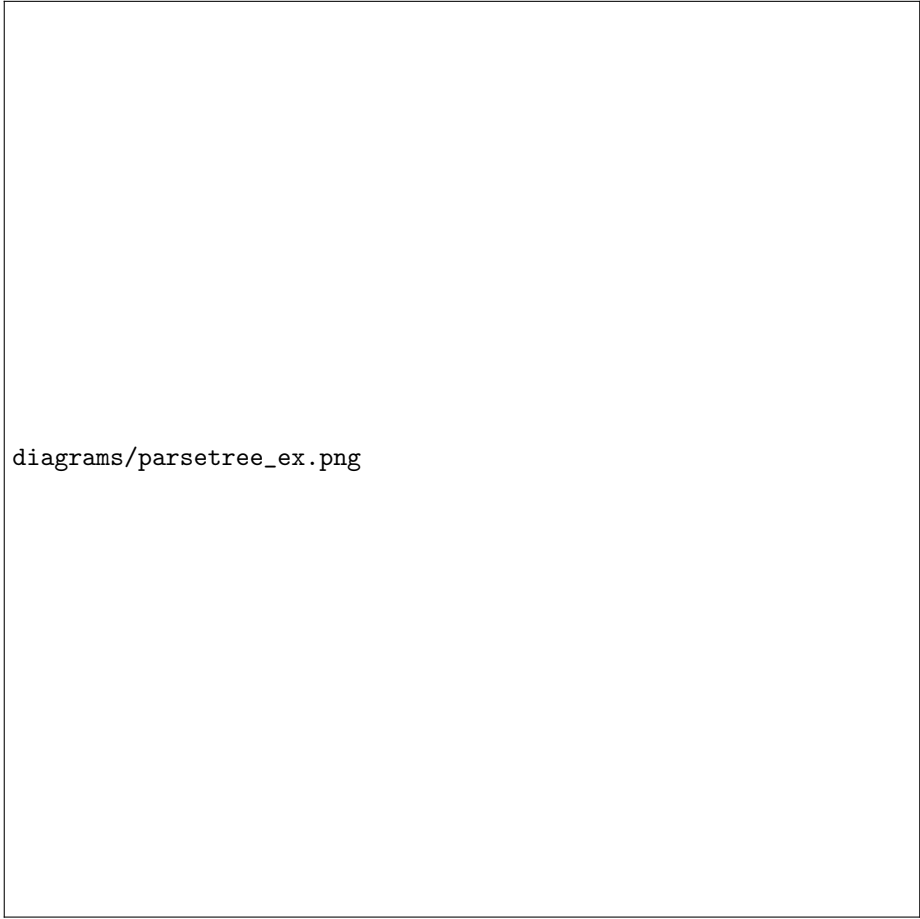
```

[ { pstr_desc =
  Pstr_value
  (Nonrecursive,
   [ { pvb_pat =
     { ppat_desc =
       Ppat_var
       { Asttypes.txt = "choice";
         loc = ...;
         ppat_loc = ... };
       pvb_expr =
         { ...
           Pexp_fun
           (Nolabel, None,
            { ...
              Ppat_var
              { Asttypes.txt = "n";
                loc = ... };
              ... },
            { ...
              Pexp_ifthenelse
              ( { ...
                Pexp_apply
                ( { Pexp_ident ...
                  { txt = Lident ">";
                    loc = ... };
                  ... } ... },
                [ (Nolabel,
                  { Pexp_ident ... { txt = Lident "n";
                    loc = ... };
                  ... } );
                ( ... { Pexp_constant
                  ( ... Pconst_integer ("0",
                    ... ) );

```

This small portion demonstrates the complexity of the AST. For ease, here is a diagram of the portions of the AST that concern our translator:

In `??`, the `Parsetree`'s root is a structure item. A `Parsetree`'s root can be a structure, signature, core type, expression or pattern. A structure is a list of



diagrams/parsetree_ex.png

Figure 3: `ParsetreeIfthenelse`

structure items. The translator only accepts structures as inputs. A structure item is an expression bound by a value.

The structure item's value is a tuple of a recursive flag and a list of value bindings. Each value-binding is a record with these fields:

```
type value_binding = {  
  pvb_pat : pattern;  
  pvb_expr : expression;  
  ...  
}
```

The example above shows that the value is bound by the string pattern “choice” and the expression of the value binding is a constructor `Pexp_fun`, a

function. This function has a variable `n` bound itself in the ternary expression `Pexp_ifthenelse`. This constructor is composed of a triple of expressions denoting the condition evaluating true/false and the two branches of ‘then’ and an option type ‘else’ expression.

The `Parsetree` type is part of the [OCaml compiler library](#) that is unstable over versions. As the language develops, the `Parsetree`’s structure changes. The use directly of this module is discouraged. Most preprocessing occurs at the `Parsetree` stage. Disparate projects attempted to create abstraction methods that would create a consistent interface with this module across versions of OCaml. These have coalesced into one library named `Ppxlib`, see ??.

The translator traverses the AST recursively. Recursive pattern-matching is used to extract portions of the tree as semantic units and convert them to a reduced AST typed by the translator.

Typedtree. The `Typedtree` is the `Parsetree` AST after typing. The `Ppxlib` `Parsetree` abstraction is typed by the module `Ocaml_common`, which is part of the compiler. This provides access to the `Typedtree`.

Recursive pattern-matching extracts the OCaml types for bound values and variables, as well as the core types for constants. These are already in string format that aligns with the `cora` signature expression.

Example 3.2. Comparison of OCaml typing and Cora signature

OCaml signature:

```
# let my_func n = n + 1;;
val my_func : int -> int
```

Cora signature:

```
my_func :: Int -> Int
```

This is collected in a hashtable with string keys of the variables and values (functions) and string representation of the typing as hashtable values. The translator then uses this to create signatures after rule production. This is a symbol table.

3.2.3 Signatures

Signatures are produced from each rule. The symbol table will then provide the signature for each term ‘call’. This occurs *after* rule creation, described further in ??.

3.2.4 TR AST

After the `Parsetree` the TR AST is defined in the `Ast_converter` module. This contains the definition of the first step of translation. The types like `tr_expr` and `tr_func` define the reduced AST of the `Parsetree`. In OCaml, these are expressed as **Algebraic Data Types** (ADT), and are called *variants*. They

are called ‘algebraic’ because of their behaviour (described in ??) as a set of *constructors*, each of which is capitalised in OCaml. ADTs/variant types are distinguished from core types (int, string, etc).

Example 3.3. Algebraic Data Types

Sum variant. The number of possibilities is determined by the sum of all possible constructors.

```
type bop =
  ...
  | TrGt
  | TrLt
  | TrGte
  | TrLte
  | TrEq
  | TrNeq
```

In the binary operator variant `bop`, we have a collection of constructors with each `bop` type expression being of one of these constructors.

Product variant. The number of possibilities is determined by the product or intersection of all elements in the collection. In OCaml, tuples and records are examples of this behaviour.

```
type tr_expr =
  ...
  | TrBinop of bop * tr_expr * tr_expr
```

In the type definition for the reduced AST, the binary operator constructor is of the product type `bop * tr_expr * tr_expr`. This is a triple comprised of a binary operator and two recursively defined binary operands.

3.2.5 Translation

In Appel’s *Modern Compiler Implementation in ML* (1998), the phases of compilation are set out [?, p.4]. After parsing the source-code there is a translation step. This takes the parsed AST and translates it to an intermediary representation (IR) that is defined internally by the program. The Translator module contains the recursive pattern-match for interacting with the `Ppxlib.Parssetree` abstraction to convert it into the `tr_ast`.

3.2.6 Canonical TR AST

Appel’s next compilation phase is the Canonical Tree [?, section 8.1]. This rewrites an AST into another AST that can be collected into a set of instructions. For Appel’s case these instructions were lines of assembly, the result of ‘flattening’ the AST into individual instructions. In this case, these are a list of rules with left-hand and right-hand terms, along with logical constraints as conditions. In the module `Canon_tree`, the concept of *tiles* from Appel’s book

is used. *Tiling* is the process of selecting *tiles* that represent the ‘block’ of code that an instruction is composed of.

Example 3.4. Tiling

Using the same code from ?? we can see the tiles that would form each rule, highlighted in different colour outlines. Righthand terms were formed from the ‘then’ and ‘else’ branches of the if and logical constraint is tiled by the binary-operator in red.

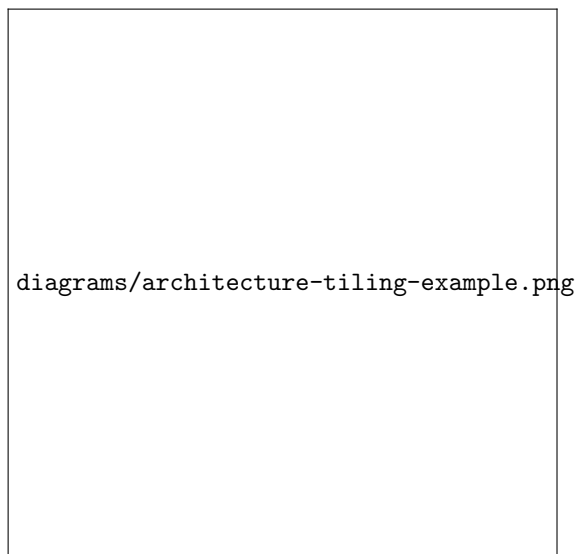


Figure 4: Tiling Ifthenelse

3.2.7 Flattening to Rules

OCaml has a language feature called pattern-matching that was used to define *tiles* as a set of exhaustive options on an `TrFunc` expression. The `Canon_tree` module uses pattern-matching to recursively traverse an expression and isolate *tiles* that would be reducible to rules, similar to the `Translator` module.

A rule is a record:

```
type rule = {
  name: string;
  lhs: term;
  rhs: term;
  cond: condition
}
```

This has been purposely designed to match the ‘triple’ within LCSTRSs of lefthand side term, righthand side term and logical condition. The fields `lhs` and `rhs` are typed as a `term` and `cond` as `condition`.

3.3 Orchestrator

The **Orchestrator** is a module that coordinates the execution of the application. It interfaces with the user and is the entry point for **Lcstrs_of_ocaml**. It applies the functions of the program to the user's inputs and outputs the **.lcstrs** artefact as well as printing to the StdOut channel.

4 Implementation

4.1 Language and File Structure

`Lcstrs_of_ocaml` was implemented in OCaml. `Dune` was used as the build-system. It allows for the declaration of external dependencies and configuration in `dune` files. The default repository structure offered by `dune` was used. `Dune` provides an abstraction to the direct compilation and dependency management of the native compiler in OCaml.

```
\src\translation
|--bin
|   |-- dune
|   |-- main.ml
|--lib
|   |--dune
|   |--ast_converter.ml
|   |--canon_tree.ml
|   |--cmd_interface.ml
|   |--orchestrator.ml
|   |--pprinter.ml
|   |--translator.ml
|   |--type_extractor.ml
|--test
|   |--dune
|   |--...
|--dune-project
--dune-workspace
```

Figure 5: Repository Structure

The `bin` directory contains a file that provides the entry point to the functionality of `Lcstrs_of_ocaml`. The directory `lib` contains the functionality of the `dune` project. Finally, the `test` directory contains the tests written for `Lcstrs_of_ocaml`.

4.2 Command Line Interface (CLI)

In `\bin\main.ml`, the `\lib\cmd_interface.ml` module is opened and it contains the ‘main’ function that is applied to inputs from the user. The `cmd_interface.ml` module interacts with the user and provides some validation on instruction. The help manual is below.

```
dune exec translate -- --help
TRANSLATE(1)                                Translate Manual                                TRANSLATE(1)
```

NAME

`translate` - Translate OCaml to LCSTRS

SYNOPSIS

`translate` `--file=FILE` `[OPTION]...` `[SRC_CODE]`

ARGUMENTS

`SRC_CODE` (`absent`=`file_input!` or `TRANSLATOR_SRCCODE` env)
Used to pass a snip of OCaml rather than a `.ml` file.

OPTIONS

`-f FILE`, `--file=FILE` (`absent`=`__default_snip`)
File option `for` providing OCaml `source` `FILE`.

COMMON OPTIONS

`--help[=FMT]` (`default`=`auto`)
...
`--version`
Show version information.

EXIT STATUS

`translate` exits with:

0 on success.
...

ENVIRONMENT

These environment variables affect the execution of `translate`:

`TRANSLATOR_SRCCODE`
Overrides Translated Source Code.

BUGS

Send bug reports to Nikos Liakas.

Translate `msc_submission`

`TRANSLATE(1)`

The external package `Cmdliner` is commonly used in dune projects to provide command line interfaces with dune projects, and was chosen by the programmer.

4.2.1 Using the command line

There are two ways to input OCaml code into `Lcstrs_of_ocaml`. The first is with a direct sample function:

```
dune exec translate "let add x y = x + y"
```

```
add :: Int -> Int -> Int
```

```
add(x, y) -> x + y
```

The second is with a file input with the `-f` optional flag⁴:

```
dune exec translate -- -f ~/add.ml
```

```
add :: Int -> Int -> Int
```

```
add(x, y) -> x + y
```

4.3 Orchestrator

The `Orchestrator` module contains the main function applied to the arguments the user inputs into to the CLI. The function applied is `tr_of_ocaml`. This function contains the high-level application path of `Lcstrs_of_ocaml`'s evaluations.

```
let tr_of_ocaml ~file src_code =
  let (ppx_ptree, comp_ptree) =

    match file with
    | "__default_snip" ->
        parse_string_both_formats src_code
    | _ ->
        read_file file |> parse_string_both_formats
    in
  let term_r_ast = parse_to_tr_ast ppx_ptree in
  let func_name =
    match term_r_ast with
    | TrAst funcs -> List.hd funcs |>
        (fun (TrFunc (_, name, _, _)) -> name)
    in
  let rules = flatten_ast_to_rules term_r_ast in
  let type_htable =
    extract_types_from_ptree_stri comp_ptree
    |> cora_types_of_types in
  let signatures = extract_rule_signatures rules type_htable in
  let output_path = "../..data/outputs/" in
  write_to_file
    (output_path ^ func_name ^ "_input.lcstrs")
    (signatures ^ "\n\n" ^ (string_of_rules rules));
  print_endline signatures;
  print_newline ();
  print_rewrite_rules rules
```

A file or source-code is parsed into a tuple comprised of:

⁴Note the two hyphens needed between the executable and the flagged arguments!

1. `ppx_ptree`: This is the `Ppxlib.Parssetree`, an abstraction over the version-specific AST of OCaml.
2. `comp_ptree`: This is the concrete and current OCaml compiler version of the `Parssetree`. See ??

Each represents the starting inputs bound by the function applications below it. A `tr_ast` type is created from parsing the `ppx_ptree`. Following that, the `tr_ast` is ‘flattened’ using the tiling method, the design of which is found in ?? and ?. This evaluates to a list of `rules`.

Starting from the `comp_ptree`, the compiler toolchain (see: ??) progresses the `Parssetree` to the `Typedtree`, see ?. This involves using the compiler to ‘type’ the AST. From this `Typedtree` AST a theory symbol table?? is derived: `type_htable`.

Both application paths are combined when the theory symbol assignment occurs using the theory symbol table `type_htable` and the `rules` to create the `signatures`.

Finally, the Orchestrator channels the resulting strings to StdOut and outputs a `.lcstrs` file in the `/data/outputs` directory.

4.3.1 Ppxlib vs Ocaml_common

Both the abstracted `Ppxlib.Parssetree` and the concrete OCaml compiler’s `Parssetree` were used. A logical question would be: why does this program need two `Parssetrees`? The answer is a direct result of the requirement that LCSTRS expressions need a *signature* for each *rule*. *Signatures* contain type information and the `Parssetree` is not typed. As ?? shows, typing occurs later in the toolchain, after the `Typedtree` phase. A well known project in the OCaml community is called `Js_of_ocaml` [?]. It compiles the OCaml Bytecode to JavaScript. The Bytecode phase is already typed and is a stack machine.

Example 4.1. OCaml Bytecode Snip

```
let square x = x * x
```

When compiled with the OCaml bytecode compiler, one can use the `ocamldebug` executable to output the following.

```
code fragment: square
0: PUSHACC 0
1: PUSHACC 0
2: MULINT
3: RETURN
```

The `Typedtree` was still syntactically OCaml source-code and could be interacted with like any other OCaml source type. In order to maintain the interoperability between different versions of OCaml, the compiler `Parssetree` was generated from the `ppx_ptree` in this way:

```

let compiler_ptree_of_ppxlib ppxlib_ptree =
  Ppxlib_ast.Selected_ast.to_ocaml Structure ppxlib_ptree

val compiler_ptree_of_ppxlib:
  Ppxlib.Parssetree.structure
  -> Ppxlib_ast.Compiler_version.Ast.Parssetree.structure
  = <fun>

```

This allows for the compiler-toolchain to be used to type the version specific `Parssetree` and return the `Typedtree`. It was difficult to find a `Typedtree` abstraction used for metaprogramming equivalent to the one that `Ppxlib` provides for the `Parssetree`. The project that was found interacted with the Bytecode compilation phase, which exceeded the scope of this project. But could be an area for future work.

4.4 Types

Types were extracted in the `type_extract.ml` module. The source for `Lcstrs_of_ocaml` types was a type called the `Typedtree`. As described in ??, the typed AST of the source-code can be recursively traversed using a recursive pattern-match for each expression node. During traversal, type values and signatures were collected to a hashtable. For an examination of the limits of this approach, see ??.

Below is an example of the `let` constructor in the `Typedtree` definition, and how it is extracted in `Lcstrs_of_ocaml`.

Example 4.2. `Typedtree let` and `Lcstrs_of_ocaml` Type-Extraction

```
| Texp_let of rec_flag * value_binding list * expression
```

This type definition for the constructor `Texp_let` is composed of a triple of:

1. `rec_flag`: A recursive flag for the `let` binding definition.
2. `value_binding list`: This is a list of value bindings.
3. `expression`: The bound value is used in this expression.

Each value binding is composed of a pattern, often a simple variable, but can be more complex, see ??. `Lcstrs_of_ocaml` is limited to simple single variable bindings. The expressions to which these patterns are bound is the second part of a single `value_bind` item in the list. Here is an example single-`let` with multiple bindings: `let a = 1 and b = 2 and c = 3`.

These are used in the final component, the body `expression`. An example using the above:

```
let a = 1 and b = 2 and c = 3 in a + b + c + a + a + a
```

The pattern-match that `Lcstrs_of_ocaml` uses first destructs the `Texp_let` node and iterates over the `value_binding list` applying a pattern type extract function and an expression type extract function to each field respectively

vb.vb_pat vb.vb_expr. As this is a sequence, syntactically ; in OCaml, the next step is a recursive application to the expression body

```
| Texp_let (_, bindings, body) ->
  List.iter (fun vb ->
    extract_from_pattern tbl vb.vb_pat;
    extract_from_expression tbl vb.vb_expr
  ) bindings;
  extract_from_expression tbl body
```

For our type extractor, there is no need to discern recursive from non-recursive bindings to define signatures.

Two functions in `type_extractor.ml` are applied either to `patterns` or `expressions`. They are respectively `extract_from_pattern` and `extract_from_expression`. As these pattern-matches expand, so does `Lcstrs_of_ocaml`'s search space within the `Typedtree` type. The control flow is quite direct, if any of these patterns are not met an internal failure is raised with `failwith` on a match case.

4.4.1 Type Table

The type hashtable produced is the collection of variable identifiers in patterns or expression identifiers in expressions. This is the pattern-match that represents an addition to the type table for both identifiers:

```
| Tpat_var (ident, _, _) ->
  let name = Ident.name ident in
  let type_str = type_to_string pat.pat_type in
  Hashtbl.replace tbl name type_str

| Texp_ident (path, _, _) ->
  let name = Path.last path in
  let type_str = type_to_string expr.exp_type in
  Hashtbl.replace tbl name type_str
```

As demonstrated, both replace or add the identifier `name` as a key and the string of type as a value. Both pairs are strings. An example of an OCaml expression and its type table is below:

Example 4.3. Type Table from Function

```
let my_func n x y =
  let z = 20 in
  string_of_int
    (n * x + y - 40 + z) ^ (
    if z > n then
      "goodbye"
    else
      "hello")
```

Lcstrs_of_ocaml's type extractor outputs the following type table:

```
{
  "^": "String -> String -> String";
  "+": "Int -> Int -> Int";
  "n": "Int";
  "-": "Int -> Int -> Int";
  "z": "Int";
  "y": "Int";
  "my_func": "Int -> Int -> Int -> String";
  "x": "Int";
  "*": "Int -> Int -> Int";
  ">": "Int -> Int -> Bool";
  "string_of_int": "Int -> String";
}
- : (string, string) Hashtbl.t
```

4.5 Signatures

At the end of Lcstrs_of_ocaml rule production phase, left-hand side terms are typed using the type table. If the type table contains the function symbol, that type declaration is retrieved from the table, as in ??:

`my_func :: Int -> Int -> Int -> String`. If a rule is created internally to the function, the left-hand side sub-terms are typed with the right-hand side term providing the terminal type κ that each mapped ι is applied, see ??. `cora` names these *private* terms.

Example 4.4. Private Rules and their Typing

```
my_func :: Int -> A
l1 :: Int -> Int -> A
l0 :: Int -> Int -> A

my_func(x) -> l0(x, 0)
l0(x, y) -> l1(x, y) | x > y
l1(x, y) -> l0(x, y + 1)
```

The signatures for `l1` and `l0` were declared by the above operation. The variables `x` and `y` were retrieved from the type table `Int -> Int`. Then the right-hand term was evaluated as a pattern-match on the type `term` defined in `Type_extractor` and a κ was applied as the `result_type`: `-> A`.

4.6 Rules

This subsection covers how ‘rules’ were implemented in Lcstrs_of_ocaml. This begins initially with parsing the `Parsetree`. As discussed in ??, Lcstrs_of_ocaml does not directly interact with the OCaml source-code, but its pre-processed state from Ppxlib, the abstract syntax tree (AST): `Parsetree`.

This AST contains much more information than needed for `Lcstrs_of_ocaml`. An example of information not needed by the project's program is that every node contains a location record with start, end and ghost as fields. To isolate the interactions, a reduced AST was translated from the `Parsetree`. As noted in ??, `Ppxlib` was used to extract a single function with arbitrarily many parameters. The module `Ast_converter` contains the reduced AST called the `tr_ast`.

```
type tr_func =

(* TrFunc (rec_flag, func_name, params, tr_body) *)
| TrFunc of string * string * string list * tr_expr

type tr_ast = TrAst of tr_func list
```

4.6.1 tr_expr

Each constructor in this recursively defined algebraic data type allowed for pattern-matching on the `Parsetree` to selectively mimic the expressions declared in the AST of the source-code.

Example 4.5. Example of `tr_expr`

```
type tr_expr =
| TrInt of int
| TrBool of bool
| TrVar of string
| TrString of string
| TrBinop of bop * tr_expr * tr_expr
...
| TrWhile of tr_expr * tr_expr
```

The `tr_expr` forms the body of the function and the recursive type definition allows for traversal of the type via pattern-matching. The sub-tree of the AST that is passed by each successive application of the function accepts the same `tr_expr`.

Example 4.6. Example of `tr_ast` of OCaml The function from ??, is restated here and converted to the reduced `tr_ast`.

```
let choice n =
  if n > 0 then
    "up"
  else
    "down"
```

The `tr_ast` has a single function `TrFunc`. This is composed of a tuple with four components: a recursion flag, the function name, a list of parameters and an

expression. This expression is recursively decomposed by the pattern-matching into the final expression. Note that `TrString` and `TrBinop` are constructors of type `tr_expr`.

```
tr_ast =
  TrAst
    [TrFunc ( "nonrec",
              "choice",
              ["n"],
              TrIfThenElse (TrBinop (TrGt, TrVar "n", TrInt 0),
                              TrString "up",
                              TrString "down"
                              )
                        )
    ]
```

4.6.2 Canon_tree and Rule Flattening

From the `tr_ast`, the reduced AST internal to `Lcstrs_of_ocaml`, we need to isolate *tiles* or sub-trees that would represent all the constituent parts of a single rule. The design of this is discussed in ???. The implementation of this is found in the module `Canon_tree`.

The canonical representation of LCSTRS is a list of rules. Each rule is typed as:

```
let rule = {
  name: string;
  lhs: term;
  rhs: term;
  cond: condition
}
```

This record represents a single rule with `lhs` and `rhs` fields as type `term` and an `cond` field as type `condition`. See ??? for design choices.

The `term` type is a recursively defined ADT like `tr_expr`. Similar to the module `Translator`, `Canon_tree` uses pattern-matching to select the *tiles* within the `tr_expr` of the function body that would commute to rules.

Here is an example of the *tiling* of an assignment on `tr_expr` node:

Example 4.7. pattern-matching `TrAssign`

This is the *tile* of the `TrBinop` node.

```
let rec flatten_to_rules func_name params body state ~continuation_name =
  match body with
  ...
| TrBinop (TrAssign, _, _) ->
  let (label_name, state1) = match continuation_name with
  Some continuation -> (continuation, state)
  | None -> new_rule_label state "1"
```

```

in
let assign_rule = create_assign_rule func_name params body label_name
in
add_rule state1 assign_rule

```

This pattern decomposes the `body` expression of type `tr_expr` for binary operators, specifically those with the `TrAssign` constructor. Assignment in OCaml is expressed using `:=` the contents. It is structured in both the `Parsetree` and `tr_expr` as a binary operator, where the pattern on the right is the left expression and the body on the right of that operator. First there is a pattern-match to regulate the control-flow if the assignment is a continuation or not. If it is not (`None`) a new label is created by flattening the `tr_expr` in the following function:

Example 4.8. Flattening the `TrAssign` node to a rule

This flattening occurs by explicitly structuring the commutation of `TrAssign` to be rewritten as the `rule` type record.

```

let create_assign_rule func_name params body label_name =
  let updates = extract_updates body in
  let updated_params = apply_updates params updates in
  {
    name = func_name ^ "_assign";
    lhs = TCall (func_name, List.map (fun p -> TVar p) params);
    rhs = TCall (label_name, updated_params);
    cond = CTrue
  }

```

The above would flatten `y := !y + 1` into this rule:

```
10(y) -> 11(y + 1)
```

4.7 Development Methodology

The project development methodology used was a loose interpretation of Agile. Work was planned in two-week sprints roughly correlated to particular OCaml language features that were aimed to be incorporated. This contrasts to Waterfall, where the language features would be thoroughly documented beforehand and each development step would follow discrete, sequential steps. These steps would be set out in a Functional and Non-functional requirement document.

Instead, the programmer iteratively expanded the feature set of `Lcstrs_of_ocaml`, while constantly testing the validity of its translations with the `cora` target syntax, by executing the analyser on the translator's outputs. The documentation of the program with `odoc` (`??`), `readme` and `appendix` sufficed for the project's development.

5 Testing and Evaluation

The testing and evaluation of `Lcstrs_of_ocaml` is described in the following subsections. The first two sections are focused on how the behaviour of the program was tested using unit-tests and the testing for `cora` valid syntax via ‘benchmarks’, respectively. The third section covers the standard by which ‘completion’ is measured for `Lcstrs_of_ocaml`’s aims.

5.1 Testing: Program Semantics

In *OCaml Programming: Correct + Efficient + Beautiful* (2025), it is stated “correct programs behave as we intend them to behave. *Validation* is the process of building our confidence in correct program behaviour” [?, 8.4]. A program’s behaviour is called its *semantics*. How we state our expectations can take a number of forms from submitting to a compiler an expression one thinks is not only syntactically valid, but will make the eventually executed code perform some expected behaviour.

Unit-tests were selected to encode expectations of the program’s semantics. Tests were written not to blindly chase a test-coverage score of 100%, but to ensure that changes made to the code-base did not introduce invalid expectations of behaviour. The two areas where unit-tests were written were: the program’s interface with the `Parsetree` in the `Translation` module, and the flattening of the internal reduced AST to rules. Those two functions present the semantic core of `Lcstrs_of_ocaml` and are the two most error-prone locations.

5.1.1 Translator module interface with `Parsetree`

The `Translator` module has a recursive pattern-match at its core, see ???. Unit-tests were written to delineate the expected behaviour of the `convert_expression` function. This was done to focus on the location in the program where the function was applied to the `Parsetree` type.

All unit-tests were written with the `dune` standard package `Alcotest`.

There are three steps to writing unit-tests with this framework.

1. Define a concrete implementation of a module that will have the type `testable`. This module contains three values and one type. Some `testable` types come standard with the package (ex: `int`).
 - 1.1. `type t = tr_expr`: The type being tested.
 - 1.2. `let rec pp format (expr:t)`: A pretty printer for debugging.
 - 1.3. `let equal a b`: A polymorphic equality function that uses pattern-matching to perform different equality tests.
 - 1.4. `let testable = testable pp equal`: A function to interface with expected and actual values under testing.
2. Write a unit-test, applying the function that one would like to test and then applying `check` from the package.

```

let test_convert_expression_binop () =
  let e1 = make_int 3 in
  let e2 = make_int 6 in
  let op_id = make_ident "+" in
  let binop_expr = {
    pexp_desc = Pexp_apply (op_id, [Nolabel, e1; Nolabel, e2]);
    pexp_loc = Location.none;
    pexp_loc_stack = [];
    pexp_attributes = []
  } in
  let result = Translator.convert_expression binop_expr in

  check
    TrExpr.testable
      "convert P-tree binop to TrBinop"
      (TrBinop(TrAdd, TrInt 3, TrInt 6))
      result

```

3. List the testcases one would like to run in one's `test/` module.

The expectation stated in this test is that the `Parsetree`'s structure is consistent with the `binop_expr` explicitly stated record. If this fails, `convert_expression` is not able to decompose this record correctly to the expected sub-tree as the third argument of `check`.

The test-suite, `test/test_suite.ml`, for `Lcstrs_of_ocaml` is run from the command line at the project's root `src/translation` using dune's `dune test`.

The unit-tests for the program's interface with `Parsetree` are located in the file `test/tr_expr_tests.ml`.

5.1.2 Rule construction from internal AST

The unit-tests for flattening the internal AST in the `tr_expr` type to rules, follow the same development path as the tests for the `Translator` module, but needed more concrete modules implemented using the `module m_name = struct ... end` structure. This was done for types `term`, `rule` and `condition`. The tests are found in `test/canon_tree_tests.ml`.

In each test the input, which has type `TrFunc`, is flattened using `Canon_tree.flatten_function_to_rules`. The expected definition is the list of rules that this input would create, written as record literals.

```

let test_flatten_let_binding () =

  let tr_expr =
    TrFunc ("non_rec", "f", ["y"],
      TrLet([("x", TrInt 1)],
        TrBinop(TrAdd, TrVar "x", TrInt 2)
      )
    )

```

```

    )
  in
  let rules = flatten_function_to_rules
              tr_expr
              init_rewrite_state
              in
  let expected_rules = List.rev rules.rules in

  match expected_rules with
  | [rule1; rule2] ->
    check Rule.testable "Let binding creates init rule"
    {
      name = "f_init";
      lhs = TCall("f", [TVar "y"]);
      rhs = TCall(
        rule1.rhs |> (function TCall(name, _) -> name | _ -> ""),
        [TVar "y"; TInt 1]);
      cond = CTrue
    }
    rule1;

    check Rule.testable "Add terminal rule"
    {
      name = rule2.name;
      lhs = TCall(
        rule1.rhs |> (function TCall(name, _) -> name | _ -> ""),
        [TVar "y"; TVar "x"]);
      rhs = TBinop(TrAdd, TVar "x", TInt 2);
      cond = CTrue
    }
    rule2

  | _ -> failwith "Expected two rules for let-binding flattening"

```

In the above code the code `let x = 1 in x + 2` is representing using `Lcstrs_of_ocaml`'s internal AST. The function is applied to it to get the actual evaluation from the function. The `Rule` module's value `testable` checks the equality of the stated two rule literals and also pattern-matches to test that the number of rules does not exceed two when that function is applied to the AST input. The other tests have similar structures.

Unit-Test Output. The module `Alcotest` provides output in this form when all tests pass:

```

dune test
Testing `lcstrs_of_ocaml_test_suite'.
This run has ID `19H1JCJH'.

```

```

[OK]    tr_expr_tests      0    convert P-tree Var to TrVar.
[OK]    tr_expr_tests      1    convert P-tree Int to TrInt.
[OK]    tr_expr_tests      2    convert P-tree [] to TrNil.
[OK]    tr_expr_tests      3    convert P-tree :: to TrCons.
[OK]    tr_expr_tests      4    convert P-tree binop to TrBinop.
[OK]    tr_expr_tests      5    convert P-tree unop to TrUnop.
[OK]    tr_expr_tests      6    convert P-tree ifthenelse to TrIfThenElse.
[OK]    tr_expr_tests      7    convert P-tree while to TrWhile.
[OK]    canon_tree_tests   0    flatten TrLet to Rules.
[OK]    canon_tree_tests   1    flatten TrAssign to Rules.
[OK]    canon_tree_tests   2    flatten TrIfThenElse to Rules.
[OK]    canon_tree_tests   3    flatten TrSequence to Rules.
[OK]    canon_tree_tests   4    flatten TrWhile to Rules.

```

Full test results in `.../_tests/lcstrs_of_ocaml_test_suite'.
Test Successful in 0.000s. 13 tests run.

When a failure occurs, the pretty printer value is applied to give the programmer a rich failure message along with the file's line number and character index.

```

[OK]            canon_tree_tests  3    flatten TrSequence to Rules.
[FAIL]          canon_tree_tests  4    flatten TrWhile to
[FAIL]          canon_tree_tests  4    flatten TrWhile to Rules.
ASSERT guard rule name
ASSERT guard LHS
File "test/canon_tree_tests.ml", line 178, character 2:
FAIL guard LHS

Expected: `TCall(f, [TVar x])'
Received: `TCall(f, [TVar n])'

```

5.1.3 OCaml: Compiler testing

OCaml has a type system derived from a language called **Meta Langauge** (ML). It is a multi-paradigm programming language from which OCaml is descended. Its design centres around a strictly typed system that would ensure 'theorem security' [?]. This system centred around an Abstract Data Type (ADT) containing instances of axioms whose operations were inference rules. The type system could then accept or reject values that could be derived/reached through inference rules, in this case theorems. ML was designed to allow for the extension and customisation of these proof commands [?].

This internal type system in OCaml, allows the compiler to eliminate whole classes of errors around type-safety. When one declares an ADT, it needs to conform to this system and prevents incomplete or unreachable types. An ex-

ample is that the programmer needs to provide ‘type holes’ explicitly stating that they have left a type as incomplete.

Example 5.1. Type Holes

```
type shape = Circle | Square | Rectangle

let find_shape s =
  match s with
  | Circle -> "circle"
  | Square -> "square"
  | _ -> "unknown"
```

The `_` case in the pattern-match is *required* by the compiler.

The above is evidence for why some type-safety tests were not written.

5.2 Testing: Cora Validation

End-to-end testing is incorporated in the project, by testing the output with the target analyser `cora`. `cora`’s rejection or acceptance of a program can provide some assurance that the expression produced is ‘valid’. The `cora` repository contains a number of benchmark inputs that it could accept as inputs. One of the key evaluating metrics is `Lcstrs_of_ocaml`’s ability to translate semantically equivalent OCaml inputs into these LCSTRS expressions.

Example 5.2. Testing `Lcstrs_of_ocaml` output with Cora

```
dune exec translate -- -f ../../data/inputs/ocaml_file_inputs/fact.ml
fact :: Int -> Int

fact(n) -> n * fact(n - 1) | n > 0
fact(n) -> 1 | n <= 0
```

And then `cora` is executed on the LCSTRS artefact.

```
cora ../../data/outputs/fact_input.lcstrs
YES
We consider termination of the LCSTRS with only rule scheme Calc:

Signature: fact :: Int → Int

Rules: fact(n) → n * fact(n - 1) | n > 0
      fact(n) → 1 | n <= 0
```

If `cora` rejects an input, `Lcstrs_of_ocaml` can be determined not to behave as expected. This works quite well as `cora` outputs detailed error messaging.

The input below declared a polymorphic type
`map :: (Int -> 'a) -> intlist -> 'a list`
for one of its inputs, which cora cannot accept yet. cora then output:

```
cora ../../data/outputs/map_input.lcstrs
../../data/outputs/map_input.lcstrs:2:16:
    Expected a type (started by a sort identifier or bracket) but got CATCHALL (').
```

5.3 Evaluation: OCaml Std Library

5.3.1 Aims

The program `Lcstrs_of_ocaml` aims to translate *any* OCaml valid syntax to cora syntax valid LCSTRS artefacts. One can say that the program has fulfilled those aims when the features in `Lcstrs_of_ocaml` can complete this translation for the OCaml Standard Library [?]. Limiting the scope to a subset of the language was the path chosen for this MSc project.

From the beginning of this project, it was understood that a translator of the entirety of OCaml's standard library would be beyond the scope of this project. In the proposal, the following language expressions were stated as goals of the translator:

1. **immutable variable assignment (non ref-type assignments)**

```
let x = 3
```

2. **branching, specifically ternary operators**

```
if x > 3 then "hello" else "goodbye"
```

3. **pattern-matching**

```
match s with
| "one" -> 1
| "two" -> 2
| _ -> failwith "Number not recognised!"
```

4. **higher-order function**

```
let increment x = x + 1
let int_list = [1;2;3;4;5]
let inc_list = List.map increment int_list
```

5. **partial application**

```
let increment = (+) 1
let int_list = [1;2;3;4;5]
let inc_list = List.map increment int_list
```

6. **recursion**

```

let rec increment i n =
  match n with
  | 0 -> i
  | _ -> increment (i + 1) (n - 1)

```

7. user defined types

```

type bop =
  Gt | Gte | Lt | Lte

```

In the ??, the success or failure of `Lcstrs_of_ocaml`'s feature set to meet these evaluation criteria is discussed with evidence for each.

6 Critical Review

This section reviews `Lcstrs_of_ocaml` by beginning with an assessment of the fulfillment of the proposed aims, see ??. This is followed by an examination of the difficulties with typing, with the central role of and pattern-matching in OCaml expressions. Finally, this section ends with a review of the problem of correctness in translation and further work.

6.1 Review of OCaml Std Library Coverage

The aims of the proposal stated in ?? are repeated here with an evaluation of the success or failure of their achievement.

1. immutable variable assignment (non ref-type assignments)

This aim was partially succesful. `let` is everywhere in OCaml. It is a value binder not a 'variable assignment' as stated above. And the value can be bound to a pattern `x` as in `let x = 42` and in the same construction can bind it to an expression, `let x = 42 in x * x`.

The latter case is implemented in `Lcstrs_of_ocaml`. As can be seen in the following from the `Translator` module:

```

let rec convert_expression expr =
  ...
| Pexp_let (Nonrecursive, vb_list, exp) ->
  TrLet(
    List.map (fun {pvb_pat=v; pvb_expr=v_expr;_} -> (
      ptree_pattern_to_string v, convert_expression v_expr)) vb_list,
    convert_expression exp
  )

```

This is a partial implementation, as it only pattern-matches on non-recursive let declarations and the decomposition only deals with simple value-bind patterns and expressions and does not perform decomposition of the `pvb_pat`. It does successfully continue the expression traversal with the recursive function application.

2. branching, specifically ternary operators

Ternary operators were easily implemented as the OCaml constructor was very general: `Pexp_ifthenelse expression * expression * option expression`. The implementation of this below.

```
let rec convert_expression expr =
  ...
  | Pexp_ifthenelse (cond, then_br, Some else_br) ->
    TrIfThenElse (
      convert_expression cond,
      convert_expression then_br,
      convert_expression else_br
    )
```

`cora` and `LCSTRS` allow for φ logical constraints. The condition expression was easily added and reversed to the final rules. Expressions that would evaluate to booleans using union operators were not implemented. Only inequalities were implemented as binary operators.

3. pattern-matching

Pattern-matching is a very large topic in OCaml. Only in one configuration was pattern-matching implemented. This was for list traversals. A very common pattern-match configuration is the following:

```
let rec inc_list l =
  match l with
  | [] -> []
  | h :: t -> h + 1 :: inc_list t
```

An integer list is recursively decomposed using the `h :: t` pattern case. This ‘cons’ operator on the left is a case where the head of a list, the first element `h` is accessed, along with the rest of the list `t`, the tail. The same syntax `::` also is an infix operator that prepends the evaluation of `h + 1` to the evaluation of the recursive assignment.

This is the only configuration that has been implemented with `TrMatch`. Here are some samples of how complex pattern-matching can be, just on one dimensional lists.

```
match l with
| [] -> []
| [x] -> one_element_left_case
| x1 :: x2 :: x3 :: t -> first_three_elements
| x1 :: (MyFavType a, MySecondFavType b, Any c) :: t -> ...
```

4. higher-order function

OCaml, like other functional programming languages, allows for passing functions as values. An example is this function:

```

let rec rec_func (f: int -> int -> int) x y =
  if x <= 0 then
    y
  else
    f x (rec_func f (x-1) y)

```

It is translated to:

```

rec_func :: (Int -> Int -> Int) -> Int -> Int -> Int

rec_func(f, x, y) -> y | x <= 0
rec_func(f, x, y) -> f(x, rec_func(f, x - 1, y)) | x > 0

```

And cora's output is

YES

We consider termination of the LCSTRS with only rule scheme Calc:

```

Signature: rec_func :: (Int → Int → Int) → Int → Int → Int

Rules: rec_func(f, x, y) → y | x <= 0
       rec_func(f, x, y) → f(x, rec_func(f, x - 1, y)) | x > 0

```

The implementation of this was necessarily partial as functional application is quite extensive in OCaml. Also of note is that, OCaml declares functions that are applied to even known inputs as returning a polymorphic type, denoted by 'a, working down the alphabet if there are different polymorphic types.

5. **partial application**

This was not implemented. It was difficult to design a control flow for recovering or substituting partially applied function values and their translation into the TRS.

6. **recursion**

Recursion was implemented partially as it is often found as a function application within the `Parsetree` at terminal points, ex an else-branch of a ternary operator. Also, as `Lcstrs_of_ocaml` limits itself to one function, any application that was not a binary operator or unary operator defined by the grammar was translated as `TrApp`. There were two outcomes when `cora` was executed. Either, there would be no signature for the function symbol and so, it would be rejected, or there would be, as the type table process would have extracted one from the initial function.

This meant recursion was equivalent to the internal application of a function that already possessed a signature. `cora` could then create the necessary nodes from the translated rules, without needing to replicate a stack for recursive calls with different arguments creating new rules.

7. user defined types

This was not attempted. `cora` is limited to a few types and although the signatures are very expressive, it was deemed unfeasible as a feature of `Lcstrs_of_ocaml`.

The above were stated in the proposal, but two other OCaml Std Library expressions are worth discussion below.

1. For Loops

One of the first translation targets attempted was an imperative style for-loop. This has been preserved in the source-code, both in interfacing with the `Parsetree` and in the rule flattening. The attempt made during development was to convert the for loop into a while loop. During development the programmer found no method of converting the incremental increment/decrement that a for loop would complete as a separate function application to a new untyped variable in a separate rule.

```
| Pexp_for (index_var, init_val, terminal_val, direct_flag, do_expr) ->
  (* Convert for loops to while loops!
   for x = 0 to 10 then [body]
   let x = ref 0 in
   while x < 10 do
     [body];
   increment x
  *)
  let for_var = ptree_pattern_to_string index_var in
  let (comp, increment) = match direct_flag with
    | Upto -> ("<", 1)
    | Downto -> (">", (-1)) in
  let condition = TrBinop (get_tr_binop_constructor comp ,
                          TrVar for_var,
                          convert_expression terminal_val) in
  TrLet ([ (for_var, TrUnop (TrRef, convert_expression init_val)) ],
        TrWhile (condition,
                  TrSequence [ convert_expression do_expr;
                               TrBinop(TrAssign, TrVar for_var,
                                       TrBinop(TrAdd, TrUnop (TrDeref, TrVar for_var), TrInt increment)
                                     ) ]))
```

2. While Loops

While loops are present in the `cora benchmarks`. They, like for loops, are rare in idiomatic OCaml programming. This was successfully implemented, but only without nesting.

```
| Pexp_while (cond, do_expr) ->
  TrWhile (convert_expression cond, convert_expression do_expr)
```

Nesting became an issue when the while loop would completely evaluate and enter another one, the second while loop would need to evaluate back

to the first position. As rules were created, the *tile* would need to have kept track of the one that came before. This would probably be solved by implementing a stack to keep track of the nesting. This was not done.

6.2 Typing and the type-system

Signature definition in LCSTRS expressions is very close to type signatures in OCaml. This allowed for fairly rapid development of extraction and definition of signatures. The other path to typing discussed briefly in ??, was to use the bytecode representation of OCaml. This would have been a more robust approach for two reasons:

1. **Reliability Over Versions.** The `Typedtree` changes with each version of OCaml. During development, the programmer switched from OCaml version 5.2.0 to 5.3.0 and a sub-node became inaccessible, because the tuple of that node added a component. The `Translator` module uses the `Ppxlib` abstracted `Parsetree` maintaining the abstraction across versions.
2. **Complexity of the Typedtree.** Type expression is fundamental to many languages, but contains a large expressive flexibility in OCaml. The implementation of types and their interactions allow for a complex ‘typed-ast’ to be formed and traversed. This is not the case with a stack-machine like expression in bytecode. This is a much simpler data structure to parse.

Finally, it is worth reviewing a fundamental aspect of OCaml: *polymorphic types*. Parametric polymorphism preceded OCaml, being developed in the 1970s by Robin Milner. This allowed for functions to be applied to different types without sacrificing type-safety. One of the requirements for any polymorphic type in `Lcstrs_of_ocaml` programs is that it include a type annotation, which are not needed for the OCaml type system to function. This limits the *automation* that the program can provide for OCaml program analysis.

```
let rec ex_01_take1 n (l: int list) ...
let rec rec_func (f: int -> int -> int) x y = ...
```

6.3 Decomposition and Composition

Decomposition and composition, sometimes called structural destruction and construction are a very useful feature in accessing constituent parts of a value.

```
let something = { x ; y ; z }      (** Construction from [x] [y] and [z] *)
let { x ; y ; _ } = something      (** Destruction, recovering [x] and [y] *)
```

As can be surmised, the expressive capacity to declare structural destruction have implications on parsing the AST that is formed during compilation in the `Parsetree`. This is not accommodated for in `Lcstrs_of_ocaml`, but is the primary method of accessing various nodes in the `Parsetree` and *tiling* the internal AST to rules.

6.4 Correctness and Translation

`Lcstrs_of_ocaml` translates OCaml to LCSTRS. As part of this project no certification or correctness verification was performed to ensure that the translated LCSTRS expression is semantically equivalent to the OCaml function that is its source.

6.5 Further work

A list of further work that can be completed on this project is below.

1. Bytecode type extraction. Use the bytecode expression of the OCaml source-code to extract types.
2. Decomposition. Incorporate a more sophisticated method of extracting decomposed values in different settings (pattern-match cases, let bindings etc).
3. More interactive CLI. The CLI could allow an inspection tool for the LCSTRS produced and attempt to create a *lineage* for the transformations.
4. Correctness Certification. Certifying correctness of `Lcstrs_of_ocaml`'s translation to OCaml source-code.

7 Conclusion

`Lcstrs_of_ocaml`'s development as a translator from OCaml source-code to Logically Constrained Simply-typed Term Rewrite System expressions is the focus of the project. The above report examined static analysis and Term Rewriting's place within it. It demonstrated the formalism's development to LCSTRS, a more syntactically accessible system that was the target of `Lcstrs_of_ocaml`'s output. The execution of this input into the program analyser `cora` and its subsequent determinations for termination analysis were shown. The design and implementation of `Lcstrs_of_ocaml` was described, and the testing and evaluation of the program's limits were discussed. Finally, the above was critically reviewed. The extent to which the aims set out in the proposal and realised by the MSc project is high.

References

- [1] A. W. Appel. *Modern Compiler Implementation in ML*. Cambridge University Press, Cambridge, United Kingdom, 1998.
- [2] F. Baader and T. Nipkow. *Term Rewriting and All That*. Cambridge University Press, Cambridge, United Kingdom, 1998.
- [3] D. Vale, C. Kop, C. Fuhs, and M. Ushakov. Cora, 2025. <https://github.com/hezzel/cora>.
- [4] M. R. Clarkson et al. OCaml Programming: Correct + Efficient + Beautiful, 2025.
- [5] Fabrice Le Fessant and Luc Maranget. Optimizing pattern matching. In Benjamin C. Pierce, editor, *Proceedings of the Sixth ACM SIGPLAN International Conference on Functional Programming (ICFP '01), Firenze (Florence), Italy, September 3-5, 2001*, pages 26–37. ACM, 2001.
- [6] OCaml Foundation. OCaml Docs.
- [7] C. Fuhs, C. Kop, and N. Nishida. Verifying Procedural Programs via Constrained Rewriting Induction. *ACM Trans. Comp. Logic*, 2017.
- [8] M. Gordon. From LCF to HOL: a short history. In *Proof, Language, and Interaction*, pages 169–186. The MIT Press, 2000.
- [9] L. Guo and C. Kop. Higher-order lctrss and their termination. In *ESOP (2)*, volume 14577 of *Lecture Notes in Computer Science*, pages 331–357. Springer, 2024.
- [10] C. Kop and N. Nishida. Term rewriting with logical constraints. In *Frontiers of Combining Systems - 9th International Symposium, FroCoS 2013, Nancy, France, September 18-20, 2013. Proceedings*, volume 8152 of *Lecture Notes in Computer Science*, pages 343–358. Springer, 2013.
- [11] Yaron Minsky, Anil Madhavapeddy, and Jason Hickey. *Real World OCaml - Functional Programming for the Masses*. O'Reilly, 2013.
- [12] X. Rival and K. Yi. *Introduction to Static Analysis: an abstract interpretation perspective*. MIT Press, Cambridge, MA, USA, 2020.

Appendices

A Github Repository

This is the link to my project's repository: [Lcstrs_of_ocaml](#)

B Testing Evidence

In this section, the results of tests evidenced in ??.

B.1 Running the test suite

After the dune project has been built, see ??, run the following command from the dune project.

```
dune test
```

This will run the test suite. The output of these tests can be seen in ??

B.2 Application Running Evidence

In this repository a few sample functions are saved for evidencing the translator run. The location is found from the root of the repository at the directory: `data/inputs/ocaml_file_inputs`

```
1. let rec ex_01_take n (l: int list) =  
    if n <= 0 then  
      []  
    else  
      match l with  
      | [] -> []  
      | h :: t -> h :: ex_01_take (n-1) t
```

```
dune exec translate -- -f ../../data/inputs/ocaml_file_inputs/ex_01_take.ml  
nil :: intlist  
ex_01_take :: Int -> intlist -> intlist  
cons :: Int -> intlist -> intlist
```

```
ex_01_take(n, l) -> nil | n <= 0  
ex_01_take(n, nil) -> nil  
ex_01_take(n, cons(h, t)) -> cons(h, ex_01_take(n - 1, t))
```

```
cora ../../data/outputs/ex_01_take_input.lcstrs
```

YES

We consider termination of the LCSTRS with only rule scheme Calc:

```
Signature: cons      :: Int → intlist → intlist
```

```

        ex_01_take :: Int → intlist → intlist
        nil        :: intlist

Rules: ex_01_take(n, l) → nil | n <= 0
      ex_01_take(n, nil) → nil
      ex_01_take(n, cons(h, t)) → cons(h, ex_01_take(n - 1, t))

```

2. `let` add x y = x + y

```

dune exec translate -- -f ../../data/inputs/ocaml_file_inputs/add.ml
add :: Int -> Int -> Int
add(x, y) -> x + y

cora ../../data/outputs/add_input.lcstrs
YES
We consider termination of the LCSTRS with only rule scheme Calc:
Signature: add :: Int → Int → Int
Rules: add(x, y) → x + y

```

3. `let rec` fact n =

```

if n > 0 then
  n * fact (n-1)
else
  1

```

```

dune exec translate -- -f ../../data/inputs/ocaml_file_inputs/fact.ml
fact :: Int -> Int

fact(n) -> n * fact(n - 1) | n > 0
fact(n) -> 1 | n <= 0
cora ../../data/outputs/fact_input.lcstrs
YES
We consider termination of the LCSTRS with only rule scheme Calc:

Signature: fact :: Int → Int

Rules: fact(n) → n * fact(n - 1) | n > 0
      fact(n) → 1 | n <= 0

```

C Getting Started

One will need to have completed the following to build the `dune` project and execute `Lcstrs_of_ocaml`.

1. Install the OCaml compiler using Opam, the OCaml package manager.
macOS:

```
brew install opam
```

Linux (Ubuntu):

```
sudo apt-get install opam
```

2. Initialize Opam and create a project switch (specific version of OCaml for this project)

```
opam init  
opam switch create 5.3.0
```

3. Install **Dune**.

```
opam install dune
```

4. Navigate from the root of the repository to

```
\src\translation
```

5. Install the project dependencies; this may take some time.

```
opam install . --deps-only
```

6. Build the project in the same directory.

```
dune build
```

7. Run with executable

```
dune exec translate -- -f path-to-file-ending-in.ml
```

or

```
dune exec translate "code-snip-in-quotations"
```

D Documentation

Documentation for the project source-code is reached by rendering in HTML.

Follow these steps to view the documentation. After building the dune project:

1. Open a terminal within the dune project and run

```
dune build @doc
```

2. Then if on a macOS or Linux execute the following command to open the HTML docs!

```
open _build/default/_doc/_html/index.html
```

These docs are navigable and interlinked with type signatures for each value with `Lcstrs_of_ocaml`.